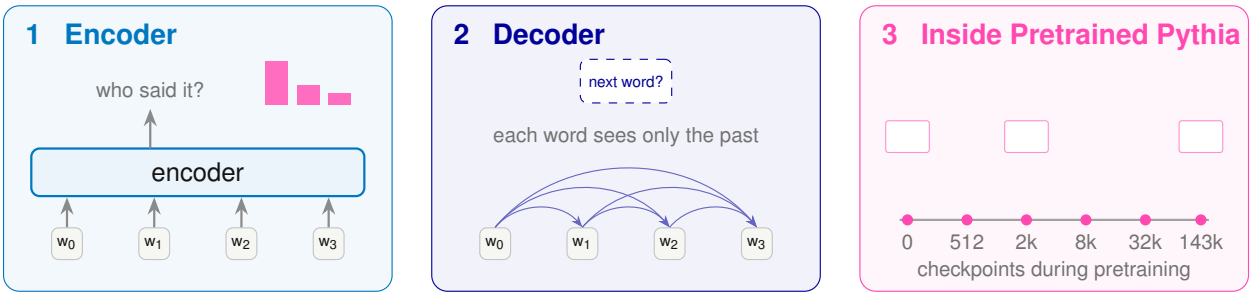
CSE 256 · Natural Language Processing · UC San Diego · Fall 2026

PA2: Transformer Blocks + Dissecting a Pretrained LLM

Build a Transformer, then open up a pretrained one and follow its checkpoints

Released: Monday, October 26, 2026 ⟶ Due: Friday, November 13, 2026 **100 points**

In this assignment you will build the two halves of a Transformer from scratch, train each on a small dataset, and then take apart a pretrained language model to see what pretraining at scale produces.



1. **Encoder.** Implement a Transformer encoder and train it jointly from scratch with a feedforward classifier that predicts which politician delivered a speech segment.[1]
2. **Decoder.** Implement a word-level, GPT-like Transformer decoder, pretrain it on next-word prediction, and report perplexity on speeches from different politicians.
3. **Inside pretraining.** Read the Pythia paper [2], then load a pretrained language model at checkpoints throughout its pretraining. Decode its hidden states layer by layer, and transplant layers between checkpoints.

Start Part 3 early. Part 3 is half the grade and can take the most wall-clock time: a single end-to-end run takes about 15–20 minutes on Colab, and you will need many runs while you explore. It does not depend on Parts 1 and 2, so you can start it first. **Part 3 is done in a Jupyter notebook and graded only from that executed notebook**: any answer, figure or number that does not appear in it receives zero credit (more details in Part 3).

Getting started. If you are new to implementing Transformer blocks, we recommend Andrej Karpathy's tutorial on building a decoder.[2] We also encourage you to read the original paper by Vaswani et al. [1].

[1] Usually the encoder is pretrained first; here you train it from scratch on the downstream task.
[2] `https://www.youtube.com/watch?v=kCc8FmEb1nY`

Libraries. Transformer blocks are implemented in libraries such as PyTorch. **Parts 1 and 2 must be implemented from scratch; no credit is given for library Transformer layers.** Part 3 has its own rules, stated there.

Preliminaries

Data

The data are speeches by three American politicians.

- **Classification**: given a short segment from a speech, predict who said it. The files are tab-separated: a label, a tab, then the segment. Labels: `0` Barack Obama, `1` George W. Bush, `2` George H. W. Bush. Train on `train_CLS.tsv`; report **accuracy** on `test_CLS.txt`.
- **Language modeling**: predict the next word. The files contain unlabeled text. Train on `train_LM.txt`; report **perplexity** on `test_LM_obama.txt`, `test_LM_wbush.txt` and `test_LM_hbush.txt`.

Starter code

Starter code is provided; you may ignore it and write your own. For Parts 1 and 2, start with the **hyperparameters in** `main.py` so that your results are comparable to ours.

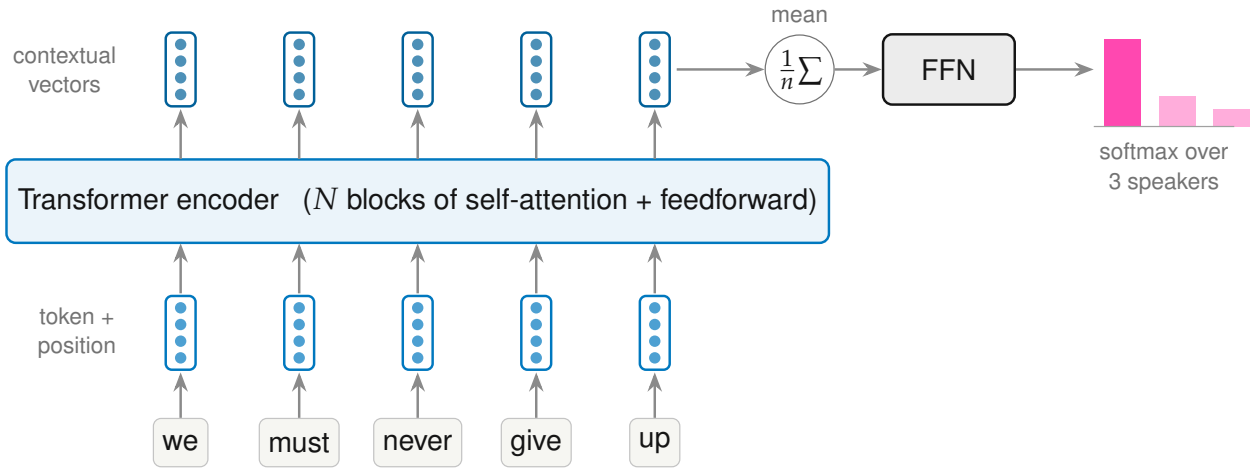
- `dataset.py`: PyTorch `Dataset` classes for both tasks.
- `tokenizer.py`: a simple word-level tokenizer (built on `NLTK`).
- `utilities.py`: sanity checks for attention: verifies that each row of an attention matrix sums to 1, and plots the matrix. Your models should return their attention matrices so you can use it.
- `main.py`: default hyperparameters and data loading; add your training loops here.
- `transformer.py`: empty. Your encoder, decoder, attention heads and feedforward layers go here.

Positional encoding and tokenizer

In Parts 1 and 2, both models use **learned absolute position embeddings**: one embedding table for tokens and one for positions, added together before the first block. Use the provided word-level tokenizer.

1 Encoder Trained with a Classifier — 30 points

You will train a Transformer encoder with a feedforward classifier on top, end to end. The encoder turns each word into a context-dependent vector; the classifier reads a summary of those vectors and predicts the speaker. **There is no pretraining in this part: encoder and classifier are trained jointly from scratch.**



1.1 Encoder

Implement the encoder with the hyperparameters in `main.py`. Its output is one vector per input word. To feed the classifier, take the **mean** of these vectors over the sequence.

1.2 Feedforward classifier

A feedforward network with one hidden layer (dimensions in `main.py`) that maps the mean vector to scores for the three speakers.

1.3 Joint training

Pass each segment through the encoder and the classifier, compute the cross-entropy loss against the true speaker, and backpropagate through *both*. The encoder learns whatever representation helps the classifier.

1.4 Sanity checks

Use `utilities.py` to check that every attention row sums to 1, and plot attention matrices for one or two sentences (any layers and heads). Describe what you see.

1.5 Evaluation

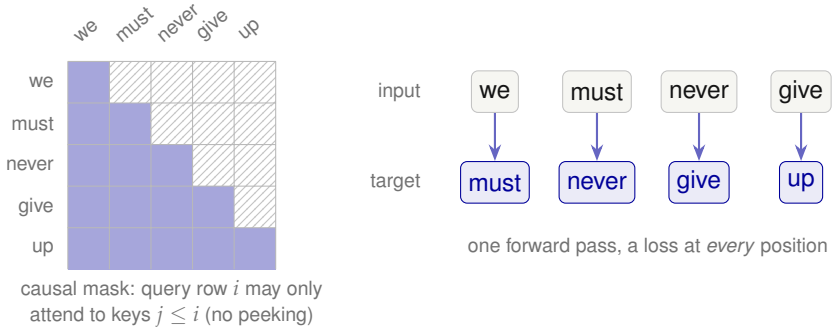
Report test accuracy on `test_CLS.txt`, after each of the 15 epochs and at the end.

> **Expected:** test accuracy in the **low to mid 80s**.

Report checklist

- Test accuracy after each epoch, and final.
- Number of parameters in your encoder.
- One or two attention-map plots, with a short discussion.

2 Pretraining a Decoder Language Model 20 points



2.1 Decoder

The decoder is like the encoder, but its self-attention is **masked** so that no position can see the future. If you forget the mask, you will notice quickly: training perplexity drops close to 1. The feedforward hidden size is 100 with ReLU; its input and output size is n_{embed}.

2.2 Pretraining

Train the decoder to predict the next word, minimizing cross-entropy over the vocabulary. In the base setting, train for **500 iterations** (batch size 16, block size 32): about 256,000 tokens. Modern language models see trillions, so do not expect low perplexities.

2.3 Sanity checks

As in Part 1: check that rows sum to 1, and plot attention matrices for one or two sentences. What does the mask look like in your plots?

2.4 Evaluation

Report perplexity on `test_LM_obama.txt`, `test_LM_wbush.txt` and `test_LM_hbush.txt`, every 100 iterations and at the end. Discuss why the three politicians get different perplexities.

> **Expected:** training perplexity in the **high 100s**; test perplexities in the **300s and 400s**.

Report checklist

- Perplexity on all three test sets every 100 iterations, and final.
- Number of parameters in your decoder.
- One or two attention-map plots, and a discussion of the differences between politicians.

3 Dissecting a Pretrained LLM — 50 points

Your decoder saw about 256 thousand tokens. LLMs see hundreds of billions to trillions. In this part you will look inside one at different moments of its pretraining. We use **Pythia** (Biderman et al. [2]), an open model family whose weights are released at many checkpoints during training. Use `EleutherAI/pythia-160m-deduped` (12 layers, 12 heads per layer, $d = 768$). Each checkpoint is a 650 MB download; Part 3 uses six of them (`step0`, `step512`, `step1000`, `step8000`, `step32000` and `step143000`), about 4 GB in total. You may use Colab, but a laptop CPU also works. On a GPU every experiment in this part runs in seconds.

You will decode a model's hidden states layer by layer, and transplant layers between checkpoints, to find out *where* it computes things and *when* during pretraining it learned to.

```
import torch
from transformers import AutoTokenizer, GPTNeoXForCausalLM
name  = "EleutherAI/pythia-160m-deduped"
tok   = AutoTokenizer.from_pretrained(name)
model = GPTNeoXForCausalLM.from_pretrained(
    name, revision="step3000",        # any checkpoint: "step0", ..., "step143000"
    torch_dtype=torch.float32)        # float16 gives NaNs late in training
out = model(ids, labels=ids,                              # out.loss, out.logits,
            output_hidden_states=True)                    # out.hidden_states
```

External Libraries. You may use `transformers` to load models and tokenizers and to generate text. All analysis code (lens readouts, transplants) must be your own, do not use interpretability libraries.

Do Part 3 in a notebook. Write Part 3 as a single Jupyter notebook (Colab is fine) and submit it *executed*, with all outputs and figures visible. Its first cell must print your name, the `torch` and `transformers` versions. Organize the notebook by question (3.1, 3.2.1, 3.2.2, 3.3), with a Markdown heading for each. **Wherever a question asks you to write something (answer, explain, compare,**

say what you find), add a Markdown text cell in your notebook and answer there, right next to the code and outputs it refers to.

> **Explanations must match your figures.** Every explanation is graded against *your own* figures and numbers: an explanation that does not match what your plot shows earns no credit, however plausible it sounds. **Results not produced in the notebook get no credit.** That includes cleared or missing outputs and values typed in by hand. We may re-run notebooks, and results that cannot be reproduced will be treated as an academic integrity issue.

3.1 Reading Pythia

4 points

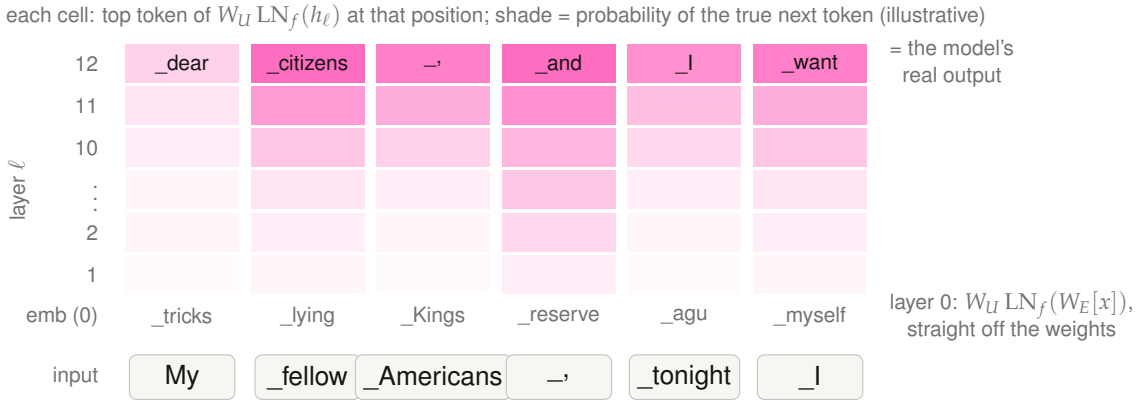
Read Biderman et al. [2] and answer each question in its own Markdown text cell in your notebook, briefly (a few sentences each), citing the relevant section from the paper.

1. **Why checkpoints?** (2 pts). Pythia releases 154 checkpoints per model, and every model size sees the training data in the *same order*. The paper presents three case studies. Which one depends most on this design, and which could you mostly do with only the final model? Explain, citing the sections of the paper.
2. **Data** (2 pts). Each Pythia size is trained twice: once on the Pile and once on a deduplicated copy of it (the "deduped" models). What does the paper find when it compares the deduped and non-deduped models? Give one finding and the section it comes from.

3.2 The Logit Lens: Reading Predictions Layer by Layer

26 points

The model makes its prediction only at the top, with logits $= W_U \, \mathrm{LN}_f(h_L)$, where W_U is the output (unembedding) matrix `embed_out` (`lm_head` in recent `transformers`; `model.get_output_embeddings()` works in both) and LN_f is the final layer norm. Nothing stops you from applying the same readout to the hidden state h_ℓ after *any* layer ℓ. This is the *logit lens* [3]: it shows what the model "would say" if it stopped computing at layer ℓ.

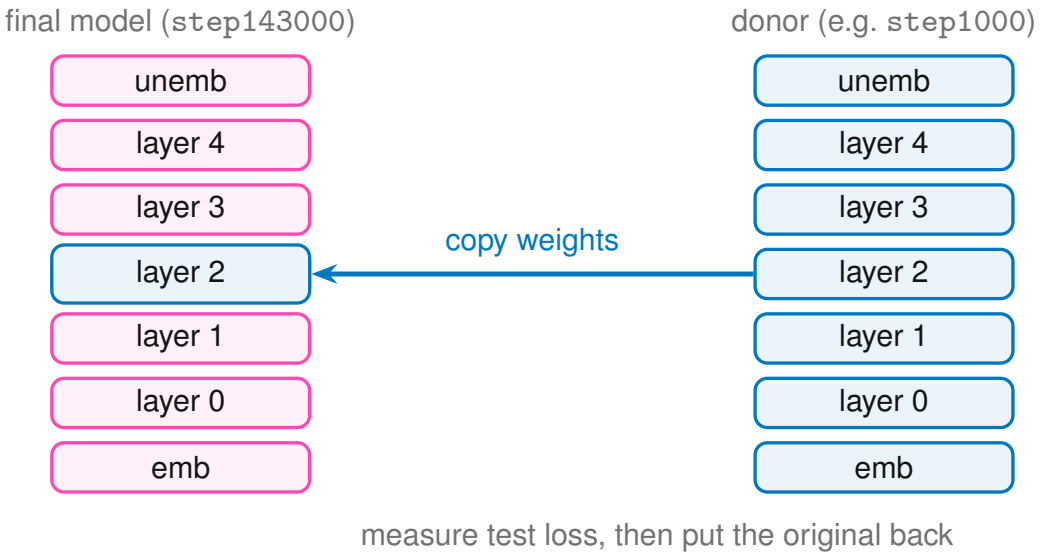


Hint. `out.hidden_states` is a tuple of $L+1$ tensors: the embeddings, then the output of each layer. Check whether the last entry already has LN_f applied (compare your readout with `out.logits`) so that you do not apply it twice.

1. **Watch a prediction form** (13 pts). Plot the layer × position grid, like the one above for *"My fellow Americans, . . . "*, at the final checkpoint for **two sentences** of about 10-15 tokens: (i) this sequence of words from `test_LM_obama.txt`: *"They have not served a Red America or a Blue America"*; (ii) a sentence you write yourself. Then, over the whole of `test_LM_obama.txt`, plot the lens's top-1 accuracy for the true next token against layer ℓ for every $\ell = 0, 1, \ldots, 12$ (13 values: the embeddings, then each of the 12 layers), and print the exact accuracy at layers 0, 6 and 12 as cell output.
2. **Across training** (13 pts). Repeat the accuracy-by-layer curve at `step512`, `step1000`, `step8000` and `step143000` on the same axes. In a Markdown text cell below the plot, answer: do the middle layers become *more* or *less* readable as training goes on? Explain what you find. What does it suggest about whether the hidden states in the middle of the network are in the same "language" as the output layer? (The *tuned lens* of Belrose et al. [4] is worth a look.)

3.3 Transplanting Layers Across Time

20 points



Pretraining changes every weight a little at a time, one gradient step after another. Are the layers of the final model still compatible with their younger selves? Take the final checkpoint and replace *one* component (the embedding, a single layer, or the unembedding) with the same component from a donor checkpoint, leaving everything else untouched. Use donors `step0`, `step1000`, `step8000` and `step32000`. For each (component, donor) pair, measure the change in test loss on `test_LM_obama.txt`. Each transplant must start from the intact final model: restore the original weights (or work on a fresh copy) before the next one. Show your results as a component × donor heatmap; use a log colour scale, since some cells are far larger than others (clip small or negative changes to the bottom of the scale), and print the exact values in the heatmap cells.

In a Markdown cell below the heatmap, answer: which components can the model lose (replaced by random `step0` weights) with little damage, and which are irreplaceable? Does a *later* donor always hurt less than an earlier one? Explain what you find, in particular any component where a later donor hurts *more*. Back every claim with a number from your heatmap or with an additional plot of your own.

Deliverables (nothing from Part 3 goes in Part A of your report):

Report checklist

- Your executed notebook as an `.ipynb` file, with all outputs.
- A PDF export of the same notebook, attached as Part B of your report. The two must come from the same run. The notebook must contain:
  - A first cell printing your name, the `torch` and `transformers` versions you use
  - Answers to the two reading questions, in Markdown text cells (3.1).
  - Logit-lens grids for your two sentences, and the accuracy-by-layer curves at four checkpoints, with your explanation in a Markdown text cell (3.2).
  - The transplant heatmap, with your explanation in a Markdown text cell (3.3).
  - The exact values asked for in each part, printed as cell output.

Submission

Submit on Gradescope.

- **Code**, with a README explaining how to run it. Ideally a single command, `python main.py`, with the default hyperparameters and options `part1` and `part2`. Part 3 lives in its notebook: include the executed `.ipynb`, with all outputs, in your code submission.
- **Report**: a single PDF with two parts.
  - **A) Parts 1–2**: about 5 pages, with reasonable font sizes. You will not be penalized for going over a bit. Use tables and figures for results rather than pasting raw output, and keep low-level code documentation in comments or the README.
  - **B) Part 3**: a PDF export of your executed Part 3 notebook, with **all cell outputs present**, attached after Part A. It must come from the same run as the `.ipynb` in your code. **All of Part 3 is graded only from this notebook.** A notebook with cleared or missing outputs, or with values not computed by its own cells, gets no credit.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017. https://papers.nips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[2] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, et al. Pythia: A suite for analyzing large language models across training and scaling. In *ICML*, 2023. https://proceedings.mlr.press/v202/biderman23a/biderman23a.pdf

[3] nostalgebraist. Interpreting GPT: the logit lens. *LessWrong*, 2020. https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens

[4] Nora Belrose, Zach Furman, Logan Smith, et al. Eliciting latent predictions from transformers with the tuned lens. *arXiv:2303.08112*, 2023. https://arxiv.org/abs/2303.08112